

Graph Machine Learning for Criminal Risk Prediction: Node Classification on the POLE-50 Dataset

Afeera Umair
Department of Computer Science
Habib University
Karachi, Pakistan

Maria Aqeel
Department of Computer Science
Habib University
Karachi, Pakistan

Mahnoor Nauman Shaikh
Department of Computer Science
Habib University
Karachi, Pakistan

Abstract—Criminal risk prediction in social networks presents a challenging machine learning problem due to severe class imbalance, small dataset sizes, and the relational nature of criminal behaviour. This paper investigates whether graph-structural features improve node classification performance over behavioural node attributes alone, using the POLE-50 dataset modelled as a property graph in Neo4j. We design four progressive feature engineering strategies and evaluate them using Logistic Regression pipelines within the Neo4j Graph Data Science library [1]. We further compare FastRP graph embeddings at three dimensionalities against Node2Vec [3] embeddings and a conventional Random Forest baseline.

I. INTRODUCTION

Criminal network analysis has traditionally depended on manual investigations and standard database searches to identify suspects and persons of interest. With the rise of graph databases and graph machine learning, investigators can now go beyond individual information and also use a person's connections and position within a network to predict criminal risk.

The POLE (Person, Object, Location, Event) data model [5] is widely used by UK law enforcement for criminal intelligence. It represents individuals, crimes, locations, vehicles, and communication records as a property graph, where relationships between entities encode investigative relevance. This structure makes it a good candidate for graph-based machine learning, where both node properties and network topology can be exploited simultaneously.

This paper addresses the research question:

Can we predict criminal risk for individuals in a social crime network, and does incorporating graph-structural features improve classification performance over node attributes alone?

We frame this as a node classification task on the POLE-50 dataset, where the target label `isInvolved` indicates whether a Person node has at least one `PARTY_TO` relationship to a Crime node.

In this paper, we will discuss the following:

- 1) We built four pipelines to compare different feature engineering strategies ranging from node attributes to graph embeddings;

- 2) we demonstrated that scalar graph metrics actively degrade performance when combined with discriminative features on a small imbalanced dataset
- 3) we showed that FastRP embeddings at 64 dimensions outperform all feature sets [2]

II. METHODOLOGY

A. Dataset and Graph Model

The POLE-50 dataset is a synthetic but realistic criminal intelligence dataset distributed by Neo4j as an example graph for forensic analysis. It was loaded into a Neo4j database instance by restoring a provided dump file, which preserved the complete graph structure, including all nodes, relationships, properties, and indexes. In total, the graph contains 61,521 nodes and 105,840 relationships across multiple entity types and interactions.

The graph contains 11 node types: Person, Crime, Location, PostCode, Area, Officer, Phone, PhoneCall, Email, Vehicle, and Object. For the purposes of this study, all analysis is scoped to Person nodes as the classification targets. The dataset contains 369 Person nodes and 29 criminally involved individuals, representing a severe class imbalance.

Relationships relevant to this study include `KNOWS` (social acquaintance), `FAMILY_REL` (family connection), `KNOWS_SN` (social network connection), `PARTY_TO` (person directly involved in a crime), `HAS_PHONE` (person linked to a phone number), `CALLER` and `CALLED` (phone call direction), and `INVOLVED_IN` (vehicle linked to a crime). Figure 1 shows the graph schema of the dataset.

B. Data Preprocessing

Prior to feature engineering, several data quality checks were performed. The graph was inspected for isolated nodes, yielding 22 isolated Vehicle nodes and 1 isolated Person node. The isolated vehicles had no impact on analysis as all pipelines were scoped to Person nodes. The isolated Person node had zero values across all features, and so this node was removed from the graph before pipeline training as it represented a data quality outlier.

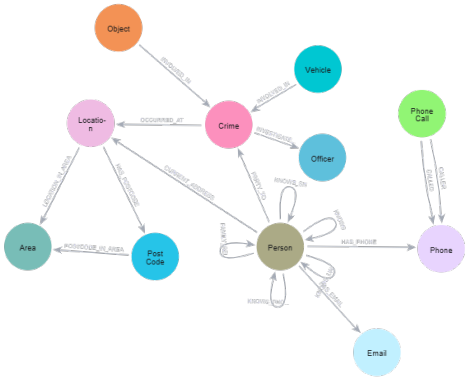


Fig. 1. Schema diagram of the Neo4j graph

No duplicate relationships were found. All Person nodes were verified to hold valid NHS number identifiers. Feature completeness was verified after each property-writing step using verification queries that confirmed no null values remained before pipeline execution.

A critical data leakage issue was identified and resolved during feature engineering. The feature `crimeCount`, representing the number of crimes a person is linked to, was initially considered as an input feature. However, since `crimeCount` is derived directly from `PARTY_TO` relationships, the same source as the `isInvolved` label, including it as a feature would allow the model to trivially read the answer. This feature was excluded from all pipeline configurations. Similarly, `uniqueCrimeTypes`, which counts distinct crime categories per person, was excluded for the same reason.

A feature separability analysis was conducted before finalising feature sets. For each candidate feature, average values were computed separately for the criminal and non-criminal classes. Features showing a class separation ratio below 1.5x were deemed to have insufficient discriminative power for a dataset with only 29 positive examples. This analysis revealed that degree-based features (`knowsDegree`, `familyDegree`, `socialDegree`), centrality measures (`betweenness` [7], degree centrality, PageRank, closeness), triangle count, and neighbourhood aggregations all showed separation ratios between 1.03x and 1.43x. Only two features demonstrated meaningful separation: `criminalVehicleCount` (0.0 vs 0.79, effectively infinite ratio) and `nightCallRatio` (0.167 vs 0.316, 1.89x ratio).

Betweenness centrality was found to have a heavily right-skewed distribution with mean 625, standard deviation 879, and maximum 5275. A log transformation was applied before inclusion in any pipeline: `betweennessLog` = $\log(\text{betweenness} + 1)$.

C. Graph Construction and Projections

Three named in-memory graph projections were created. The `social` projection contains Person nodes connected by undirected `KNOWS` relationships and was used for betweenness centrality and Louvain [4] community detection. The

`crime-associates` projection uses the same topology and was used for triangle counting. The `social-full` projection includes Person nodes connected by undirected `KNOWS`, `FAMILY_REL`, and `KNOWS_SN` relationships, and includes all node properties as named features for ML pipeline training.

The `social-full` projection was constructed in two phases. In the first phase it was projected without the embedding property, which did not yet exist. FastRP embeddings were then generated and written to nodes. In the second phase the projection was dropped and recreated with the embedding property included, making it a pipeline feature.

D. Feature Engineering

All features were written as persistent node properties using Cypher SET queries. Table I summarises all engineered features.

TABLE I
ENGINEERED NODE FEATURES

Feature	Type	Source
<code>isInvolved</code>	Label (0/1)	<code>PARTY_TO</code> traversal
<code>knowsDegree</code>	Scalar	<code>KNOWS</code> degree count
<code>familyDegree</code>	Scalar	<code>FAMILY_REL</code> degree count
<code>socialDegree</code>	Scalar	<code>KNOWS_SN</code> degree count
<code>betweennessLog</code>	Scalar	GDS betweenness + log transform
<code>triangleCount</code>	Scalar	GDS triangle count
<code>communityId</code>	Categorical	GDS Louvain
<code>criminalNeighborCount</code>	Scalar	2-hop <code>KNOWS</code> to Crime
<code>criminalFamilyCount</code>	Scalar	<code>FAMILY_REL</code> to criminals
<code>nightCallRatio</code>	Scalar	Phone calls 00:00–05:00
<code>criminalVehicleCount</code>	Scalar	Vehicles shared with crimes
<code>embedding</code>	Vector (64-dim)	FastRP

The target label `isInvolved` was derived by traversing the graph: each Person node was assigned 1 if it had at least one outgoing `PARTY_TO` relationship to a Crime node, and 0 otherwise. This derivation from graph structure rather than an external attribute file is itself a demonstration of graph-native data preparation.

The `nightCallRatio` feature was computed as the proportion of outgoing phone calls made between midnight and 5am, motivated by our Milestone 1 queries showing that late-night call activity correlates with criminal coordination in this dataset. The `criminalVehicleCount` feature counts vehicles linked to crimes that the person was also party to, capturing direct crime-scene vehicle involvement.

FastRP embeddings were generated using the `social-full` projection with embedding dimension 64 and random seed 42. Embeddings were generated transductively on the full graph prior to the train-test split, which is standard practice for fixed closed graphs but would not generalise to an inductive setting with unseen nodes.

E. Node Classification Pipelines

All classification experiments were conducted using the Neo4j GDS node classification pipeline API. Each pipeline follows a five-step configuration: creation, classifier addition,

feature selection, train-test split configuration, and training with evaluation.

All pipelines used a test fraction of 0.2 and 5-fold cross-validation on the training set. The random seed was fixed at 42 for reproducibility. Four penalty values (0.001, 0.01, 0.1, 1.0) were added to each Logistic Regression pipeline and GDS automatically selected the best via cross-validation.

Four primary pipelines were designed with progressively richer feature sets:

Pipeline A – Behavioural Features: nightCallRatio, criminalVehicleCount. This represents the strongest scalar baseline identified through feature separability analysis.

Pipeline B – Behavioural + Structural: All of Pipeline A plus betweennessLog, triangleCount. Tests whether GDS-computed graph structural metrics improve over the behavioural baseline.

Pipeline C – Behavioural + Structural + Neighbourhood: All of Pipeline B plus criminalNeighborCount, criminalFamilyCount. Tests whether indirect criminal exposure adds further predictive value.

Pipeline D – Graph Embeddings: FastRP embedding vector only. Tests whether holistic graph representation outperforms all hand-crafted feature sets.

To evaluate embedding algorithm sensitivity, Node2Vec embeddings were also generated and evaluated as Pipeline E. To evaluate embedding dimensionality sensitivity, FastRP was tested at 32, 64, and 128 dimensions.

F. Evaluation Metrics

Three metrics were reported for all experiments. Accuracy is included for completeness but is misleading given the 92.4 percent majority class – a classifier predicting non-criminal for all nodes achieves 92.4 percent accuracy without learning anything. F1-Weighted accounts for class frequency but remains biased toward the majority class. F1-Macro computes F1 separately for each class and averages equally, treating both classes as equally important regardless of size. F1-Macro is the primary evaluation metric throughout this study.

III. RESULTS

A. Feature Separability Analysis

Table II presents the class-conditional averages for key features. The analysis confirms that degree-based and centrality features show minimal separation between classes, while criminalVehicleCount and nightCallRatio show the strongest separation among scalar features.

B. Pipeline Classification Results

Table III presents the test set performance of all pipeline configurations.

C. Embedding Dimensionality

FastRP performance varied substantially across embedding dimensions. At 32 dimensions the model collapsed to majority class prediction (F1-Macro 0.4861), identical to the uninformative baseline. At 64 dimensions the model achieved

TABLE II
FEATURE CLASS-CONDITIONAL AVERAGES

Feature	Non-Criminal	Criminal	Ratio
knowsDegree	3.07	4.38	1.43x
betweenness	598.8	932.1	1.56x
triangleCount	0.67	1.03	1.54x
criminalNeighborCount	0.29	1.03	3.55x
criminalFamilyCount	0.06	0.55	9.17x
nightCallRatio	0.167	0.316	1.89x
criminalVehicleCount	0.0	0.79	∞

TABLE III
NODE CLASSIFICATION RESULTS

Pipeline	Features	Acc.	F1-W	F1-M
A (LR)	Behavioural	0.9595	0.9335	0.6895
B (LR)	+ Structural	0.9459	0.8958	0.4861
C (LR)	+ Neighbourhood	0.9459	0.8958	0.4861
D-32 (LR)	FastRP 32-dim	0.9459	0.8958	0.4861
D-64 (LR)	FastRP 64-dim	0.9595	0.9467	0.7751
D-128 (LR)	FastRP 128-dim	0.93	0.92	0.70
E (LR)	Node2Vec 64-dim	0.9459	0.8958	0.4861

peak performance of 0.7751 F1-Macro. At 128 dimensions performance degraded to 0.70, suggesting mild overfitting. This demonstrates that 64 dimensions represents the optimal balance between representational capacity and the available training signal for a graph of 369 nodes.

IV. DISCUSSION

A. Scalar Features Actively Degrade Performance

We found that Pipelines B and C perform worse than Pipeline A. Adding structural features (betweenness, triangle count) and neighbourhood features (criminal neighbour count, criminal family count) to the two effective behavioural features does not only fail to help, it returns performance to the uninformative baseline of 0.4861 F1-Macro. This suggests that on a severely imbalanced dataset with only 29 positive training examples, additional scalar features introduce noise dimensions that overwhelm the discriminative signal from nightCallRatio and criminalVehicleCount. A linear classifier with insufficient positive examples cannot reliably identify which dimensions are informative when given many weakly separating features alongside strongly separating ones.

This finding has practical implications: feature selection based on empirical class separability analysis, as conducted in this study, is essential for small imbalanced graph datasets. Naively adding more graph features degrades rather than improves performance.

B. Embeddings Capture What Scalars Cannot

Pipeline D-64 achieves F1-Macro of 0.7751, a 12.4 percent relative improvement over the scalar ceiling of 0.6895. The FastRP embedding encodes multi-hop neighbourhood structure (who a person knows, who those people know, and so on across multiple hops) into a dense 64-dimensional vector. This

avoids the noise accumulation problem of individual scalar features because the embedding algorithm implicitly weights structural dimensions by their variance across the graph. The linear classifier then operates in an embedding space where criminal and non-criminal nodes are geometrically separable, as evidenced by Logistic Regression outperforming Random Forest on the same embeddings.

C. Node2Vec Fails on Small Sparse Graphs

Node2Vec embeddings collapsed to majority class prediction regardless of parameter configuration. This is likely due to the graph's small size and sparse connectivity. Node2Vec learns embeddings by training on random walks through the graph, which requires sufficient structural diversity to produce meaningful representations. In a graph with only 369 nodes and limited connectivity, the random walks may repeatedly traverse similar neighbourhoods, reducing the quality of the learned embeddings. FastRP's matrix-based approach appears more robust under these conditions. This suggests that for criminal intelligence graphs of this scale, random projection methods may be more suitable than random-walk-based neural embedding methods.

D. Community Detection as a Feature

The Louvain algorithm identified community 91 as containing 8 of 29 criminals (27.6 percent of all criminal nodes), confirming that community membership correlates with criminal involvement. However, communityId as a raw integer feature was not usable in Logistic Regression because averaging community IDs is mathematically meaningless; community 91 and community 233 are categorical labels, not ordinal values. In future work, community membership could be one-hot encoded or used to generate community-level aggregate features.

E. Class Imbalance as the Fundamental Constraint

All results must be interpreted in the context of severe class imbalance: 340 non-criminals versus 29 criminals (7.6 percent positive rate). With an 80/20 train-test split, the test set contains approximately 6 criminal nodes. This means F1-Macro scores have high variance as a single misclassification at the boundary can shift results substantially. The plateau at F1-Macro 0.4861 across multiple pipeline configurations represents the majority class classifier baseline: a model that predicts non-criminal for every node achieves this score without learning anything about criminal structure.

V. FUTURE WORK

Several directions would strengthen this analysis. First, applying this methodology to a larger criminal network dataset with hundreds of thousands of nodes would allow Node2Vec and higher-dimensional embeddings to be properly evaluated and would reduce the variance introduced by having only 29 positive examples. Second, implementing an inductive embedding approach i.e. generating embeddings for test nodes independently of training nodes, would address the transductive leakage limitation noted in the methodology. Third,

community membership could be incorporated as a properly encoded categorical feature rather than a raw integer, potentially recovering discriminative signal from the Louvain output. Fourth, graph neural network approaches such as GraphSAGE or Graph Attention Networks could be evaluated as alternatives to the GDS pipeline framework, potentially capturing richer structural patterns than FastRP. Fifth, SMOTE [6] or other oversampling techniques applied outside GDS could address the class imbalance more directly than the current approach.

VI. CONCLUSION

This paper investigated criminal risk prediction as a node classification task on the POLE-50 dataset using the Neo4j Graph Data Science library. We evaluated four progressive feature engineering strategies and multiple embedding configurations, yielding four principal findings.

First, a systematic feature separability analysis identified only two scalar features with meaningful discriminative power: `criminalVehicleCount` and `nightCallRatio`, with class separation ratios of effectively infinite and 1.89x respectively. All other features including degree counts, centrality measures, and neighbourhood aggregations showed insufficient separation.

Second, adding structural and neighbourhood graph features to the effective behavioural baseline actively degraded performance, returning F1-Macro from 0.6895 to 0.4861. This demonstrates that feature quantity is counterproductive on small imbalanced datasets. Noise accumulation from weakly discriminative features overwhelms the signal from strongly discriminative ones.

Third, FastRP embeddings at 64 dimensions achieved the best overall performance of 0.7751 F1-Macro, a 12.4 percent relative improvement over the scalar ceiling. Embedding dimensionality proved critical: 32 dimensions collapsed to baseline, 64 was optimal, and 128 showed mild overfitting.

Fourth, Node2Vec embeddings failed entirely on this dataset, collapsing to majority class prediction regardless of parameter configuration. This is attributed to insufficient graph density and size for the random walk neural training process to converge meaningfully.

Together these findings demonstrate that for small sparse criminal network graphs, graph embeddings via random projection methods represent the most effective approach to node classification, and that individual scalar graph metrics should be selected empirically rather than assumed to be informative.

REFERENCES

- [1] Neo4j, Inc., "Neo4j Graph Data Science Library," *Neo4j Documentation*, 2023. [Online]. Available: <https://neo4j.com/docs/graph-data-science/current/>
- [2] H. Chen, S. F. Sultan, Y. Tian, M. Chen, and S. Skiena, "Fast and Accurate Network Embeddings via Very Sparse Random Projection," in *Proc. 28th ACM Int. Conf. Information and Knowledge Management (CIKM)*, 2019, pp. 399–408.
- [3] A. Grover and J. Leskovec, "node2vec: Scalable Feature Learning for Networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 855–864.

- [4] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, p. P10008, 2008.
- [5] Neo4j, Inc., "POLE Crime Dataset," *Neo4j Graph Examples*, 2022. [Online]. Available: <https://github.com/neo4j-graph-examples/pole>
- [6] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [7] U. Brandes, "A Faster Algorithm for Betweenness Centrality," *Journal of Mathematical Sociology*, vol. 25, no. 2, pp. 163–177, 2001.